Regimes of forecast error

Clustering the outcomes of the ESS experiments: idea, mathematics, and how it plugs into the code

Design document, companion to `model_structure.pdf`

Status: design + exploratory prototype (`docs/prototypes/regimes_prototype.py`); not yet in the experiment package

2026-10-27

The idea in three sentences. Every experiment draw ends in one of a small number of market outcomes, because the battery's day-ahead schedule only changes when the forecast crosses a decision threshold. Grouping the draws by that outcome gives *regimes*: each has a mechanism (what went wrong), a loss, and a probability that depends on the error parameters. The mean welfare loss is then *exactly* $\sum_k p_k L_k$, so every number of the experiments can be explained as "how often each kind of mistake happens × how much it costs".

Contents

1 The idea in one picture **2**

2 Why cluster by outcome, not by error shape **2**

3 The mathematics **3**

3.1 The chain, at a glance . . . 3
3.2 Draws and settings . . . 3
3.3 The outcome map and the regimes . . . 3
3.4 The loss is a regime constant, so the mean decomposes exactly . . . 3
3.5 Estimation . . . 3
3.6 Comparing two settings: transitions under common random numbers . . . 4
3.7 Naming the regimes: mechanism features . . . 4
3.8 Larger systems: when the loss is not constant within a signature . . . 4
3.9 From model probabilities to "how often in reality" . . . 4

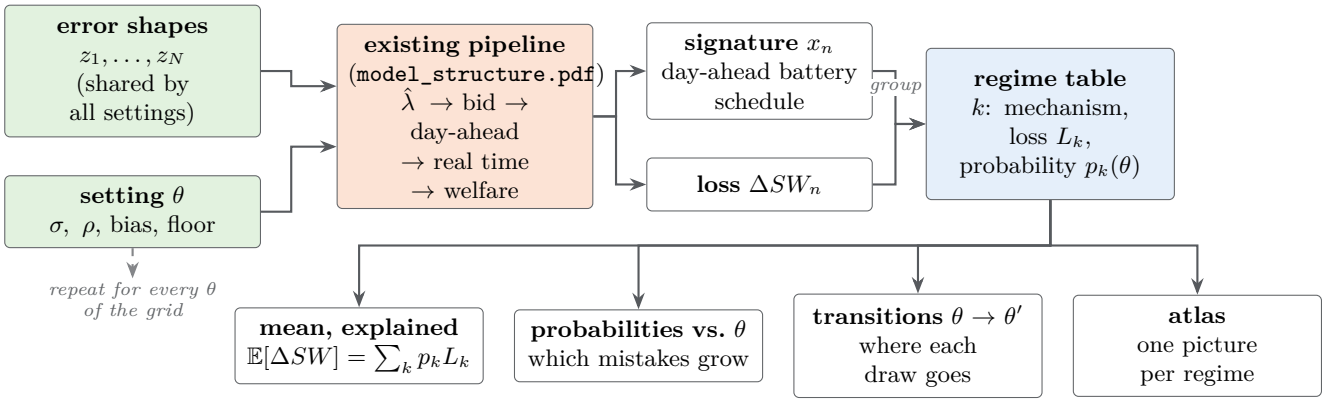
4 First look: the prototype **4**

5 Code topology: how it plugs into the existing code **5**

5.1 What is new and what changes . . . 6
5.2 Data flow of one regime run . . . 7
5.3 Interfaces (sketch) . . . 7
5.4 Acceptance tests . . . 7
5.5 Work plan . . . 7

6 Limitations to state **7**

1 The idea in one picture



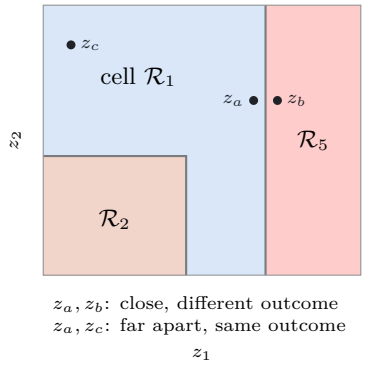
Nothing in the model changes: the regimes are a way of *reading* the experiment outputs. Two ingredients are new: (i) the same error shapes z_n are reused for every setting (common random numbers), and (ii) every draw records its signature next to its loss.

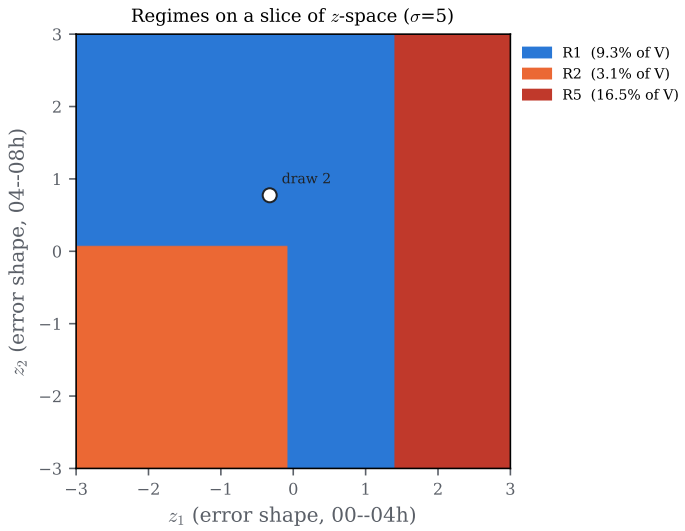
What it adds to the thesis.

- **Explains the mean.** A rise of the mean loss with σ becomes "regime k becomes more likely": a mechanism, not only a number.
- **Shows the tail.** A rare, expensive regime is visible with its probability, instead of being averaged away.
- **Answers "how often does this happen".** Each regime carries a probability under the error model; with calibrated or empirical errors (stage 3) that becomes a frequency in a real market.
- **Cleaner comparisons.** With common random numbers, the difference between two settings is the net flow of draws between regimes, with far less noise than two independent samples.

2 Why cluster by outcome, not by error shape

The error shape $z \in \mathbb{R}^T$ is the natural thing to cluster, but it is the wrong one. The market outcome is a *piecewise-constant* function of z: z-space is cut into cells by threshold hyperplanes, and inside a cell nothing in the market changes. Two shapes that are close but on opposite sides of a threshold give different outcomes; two that are far apart in the same cell give the same one. Clustering z (e.g. k-means) would mix cells; clustering the outcome recovers them exactly.

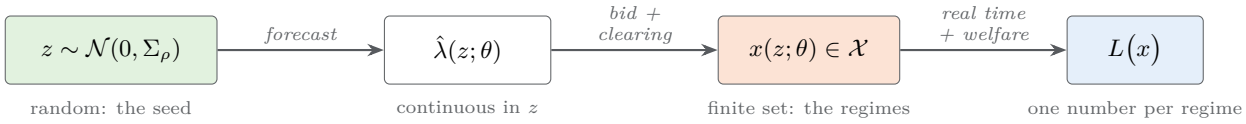




Left: the principle. Right: the real partition of the test system on a 2-D slice of z-space (z_1, z_2 vary, the other four intervals fixed at the values of a typical draw, $\sigma = 5$ \$/MWh). The boundaries are straight lines, because a threshold on the forecast $\hat{\lambda}_t = \lambda^\star_t + \sigma z_t$ is a threshold on z_t. Crossing $z_1 \approx 1.4$ (a forecast about 7 \$/MWh too high at 00–04h) moves the draw from $\mathcal{R}_1$ (9.3% of V) to $\mathcal{R}_5$ (16.5%).

3 The mathematics

3.1 The chain, at a glance



3.2 Draws and settings

One battery, T intervals. A draw is an error shape $z \in \mathbb{R}^T$, Gaussian with the AR(1) covariance

$$z \sim \mathcal{N}(0, \Sigma_\rho), \qquad (\Sigma_\rho)_{ts} = \rho^{|t-s|},$$

(with one battery the common/specific split ω has no effect). A setting $\theta = (\sigma, \rho, b, \underline{\lambda}, \text{rule}, \psi)$ turns it into a forecast (`model_structure.pdf`, §5):

$$\hat{\lambda}(z;\theta) = \max\big(\underline{\lambda},\ \lambda^\star + b\,\bar{\lambda} + \sigma z\big).$$

ρ enters through Σ_ρ; in the code the same random numbers produce z for every ρ, so common random numbers also hold across ρ.

3.3 The outcome map and the regimes

Let $x(z;\theta)$ be the battery's day-ahead schedule $(p_t^{da}, g_t^{da})_{t=1}^T$ produced by bid construction and clearing (M3–M5). Every LP in that chain returns a vertex solution, so x takes values in a *finite* set $\mathcal{X} = \{x_1, x_2, \dots\}$, and is constant between thresholds. A **regime** is the set of error shapes that produce the same schedule:

$$\mathcal{R}_k(\theta) = \{z \in \mathbb{R}^T :\ x(z;\theta) = x_k\}, \qquad \bigcup_k \mathcal{R}_k(\theta) = \mathbb{R}^T, \quad \mathcal{R}_k \cap \mathcal{R}_l = \emptyset\ (k \neq l).$$

Its probability under the error model is $p_k(\theta) = \Pr\big(z \in \mathcal{R}_k(\theta)\big)$, with $\sum_k p_k = 1$.

3.4 The loss is a regime constant, so the mean decomposes exactly

Given the day-ahead schedule, everything after it is deterministic and uses only the true system: the real-time repair (M6) takes (p^{da}, g^{da}), the redispatch (M7) takes the generators' day-ahead dispatch, which the hourly clearing fixes from the net demand $D_t + p_t^{da} - g_t^{da}$, and welfare is evaluated at the true parameters. Hence the loss depends on z only through the regime:

$$\Delta SW(z;\theta) = L\big(x(z;\theta);\psi\big) = L_k \quad \text{for } z \in \mathcal{R}_k(\theta),$$

and the moments of the loss are exact finite sums:

$$\boxed{\mathbb{E}[\Delta SW \mid \theta] = \sum_k p_k(\theta)\, L_k, \qquad \mathrm{Var}[\Delta SW \mid \theta] = \sum_k p_k(\theta)\big(L_k - \mathbb{E}[\Delta SW \mid \theta]\big)^2}$$

The argument needs the generators' day-ahead dispatch to be unique given the net demand; with tied generator costs it could depend on the bids too. The prototype checks the claim directly: across 2 400 draws the largest loss difference within a regime is **0.0** (§4).

3.5 Estimation

With N draws $z_1, \dots, z_N$ and $n_k = \#\{n : x(z_n;\theta) = x_k\}$:

$$\hat{p}_k = \frac{n_k}{N}, \qquad \mathrm{se}(\hat{p}_k) = \sqrt{\frac{\hat{p}_k(1-\hat{p}_k)}{N}}, \qquad \hat{\mu} = \sum_k \hat{p}_k L_k = \frac{1}{N}\sum_n \Delta SW(z_n;\theta).$$

The decomposed mean is *identical* to the Monte Carlo mean (not an approximation); the regime table only re-arranges it. Regimes never seen in the sample have $\hat{p}_k = 0$; the chance of missing a regime of probability p is $(1-p)^N$ (0.7% for $p = 1\%$ at $N = 500$).

3.6 Comparing two settings: transitions under common random numbers

Use the *same* z_n under θ and θ'. The transition matrix

$$P_{kl}(\theta \to \theta') = \frac{1}{N} \#\{n : \; x(z_n;\theta) = x_k, \; x(z_n;\theta') = x_l\}$$

has row sums $\hat{p}_k(\theta)$ and column sums $\hat{p}_l(\theta')$, and splits the change in mean loss into flows between regimes:

$$\hat{\mu}(\theta') - \hat{\mu}(\theta) = \sum_{k,l} P_{kl} \left(L_l - L_k\right).$$

Draws that stay in their regime ($k = l$) contribute nothing, which is why common random numbers make the comparison so much less noisy: only the draws that *cross a threshold* carry the difference.

3.7 Naming the regimes: mechanism features

A regime key is a long vector of MW; for reading it is described by a few features of its outcome:

Feature	Definition	Reads as
pattern	per interval: C (charges), D (discharges), – (idle), day-ahead	what the battery sold
E^{free}	$\Delta T \sum_{t:\lambda_t^\star = 0} p_t^{da}$	did it buy the free energy?
infeasible intervals	$\#\{t : s_t^{da} \notin [\underline{s}, \bar{s}]\}$	how wrong the day-ahead plan is
undeliverable	$\Delta T \sum_t \lvert p_t - p_t^{da}\rvert + \lvert g_t - g_t^{da}\rvert$	how much real time must correct
shed	$\Delta T \sum_t sh_t$	did it cause load shedding?
thermal at cheap hours	$\Delta T \sum_{t:\, p_t^{da} > 0} q^{rt}_{\text{thermal},t}$	did it charge from expensive supply?

A short rule-based label is generated from them ("missed morning sale", "charges from thermal at 00–04h", ...); the label is for reading only, the regime is still defined by the exact schedule.

3.8 Larger systems: when the loss is not constant within a signature

With 24 intervals, ramps, several batteries or a network, (i) exact schedules may almost never repeat, and (ii) the real-time outcome can depend on more than the day-ahead battery schedule. Then:

- use a **coarse signature** (the feature vector above, rounded) or cluster the standardised feature vectors with k-medoids, choosing K by silhouette;
- the decomposition still holds with *conditional* means, $\mathbb{E}[\Delta SW] = \sum_k p_k \, \mathbb{E}[\Delta SW \mid k]$, and the within-regime spread $\mathrm{Var}[\Delta SW \mid k]$ is reported next to each regime as a quality measure of the clustering.

3.9 From model probabilities to "how often in reality"

$p_k(\theta)$ is a probability *under the error model*. Two routes make it a real-market frequency (stage 3):

1. **Calibrate, then sample:** estimate σ, ρ, bias from a backtest of real forecasts and read p_k at the calibrated θ.
2. **Empirical shapes:** replace the Gaussian bank by real forecast-error days z_n^{emp} (`forecasts.bootstrap_days`); then $\hat{p}_k$ is directly the share of real days in regime k. Existing Gaussian draws can also be re-weighted to an empirical density f with importance weights:

$$w_n = \frac{f(z_n)}{\varphi_{\Sigma_\rho}(z_n)}, \qquad \hat{p}_k = \frac{\sum_n w_n \, \mathbf{1}[z_n \in \mathcal{R}_k]}{\sum_n w_n}.$$

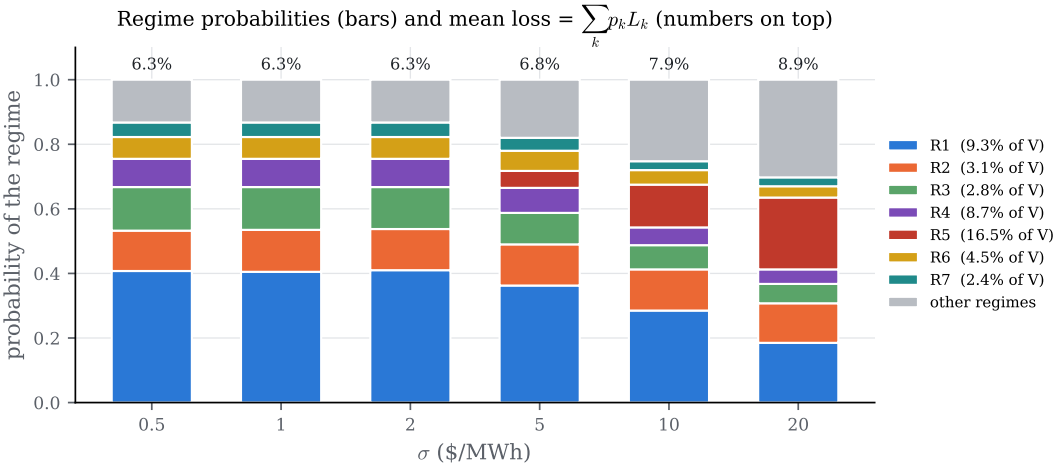
4 First look: the prototype

`docs/prototypes/regimes_prototype.py`: test system `day_case`, `B_fcst`, prelimit rule, $\psi = 10$, price floor 0, $\rho = 0$; $N = 400$ error shapes (seeds $0, \ldots, 399$) reused for every $\sigma \in \{0.5, 1, 2, 5, 10, 20\}$ \$/MWh. These are exploratory numbers that show the method works; the real runs follow §5.

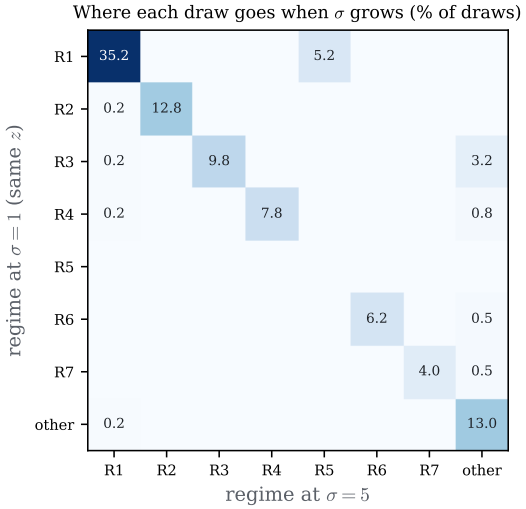
Checks. Loss constant within every regime (largest spread 0.0); $\sum_k \hat{p}_k L_k$ equals the Monte Carlo mean at every σ (e.g. 6.8132 = 6.8132 at $\sigma = 5$); 31 distinct regimes in total, 14–28 per σ; the 7 most frequent cover 87% of draws at $\sigma = 1$ and 70% at $\sigma = 20$; the transition flows between $\sigma = 1$ and $\sigma = 5$ add up exactly to the change in mean loss (+0.558).

	pattern	loss	probability at $\sigma =$			undeliv.	mechanism
	(00h → 24h)	(% of V)	1	5	20	(MWh)	
$\mathcal{R}_1$	C-CCDD	9.31	0.405	0.362	0.185	1 371	keeps the morning charge, no 04–08h sale
$\mathcal{R}_2$	CDCCDD	3.09	0.130	0.128	0.122	1 051	benchmark-like cycle, oversells the evening
$\mathcal{R}_3$	C-CDD	2.83	0.133	0.098	0.060	580	charges in only one free interval
$\mathcal{R}_4$	C-CCDD	8.70	0.087	0.077	0.045	1 071	as $\mathcal{R}_1$, but sells less at 16–20h
$\mathcal{R}_5$	C-CCDD	16.46	0	0.052	0.223	1 731	buys 200 MW from thermal at 00–04h
$\mathcal{R}_6$	C-CCD-	4.47	0.068	0.062	0.035	1 071	holds energy to the end of the day
$\mathcal{R}_7$	CD-CDD	2.36	0.045	0.040	0.028	420	sells in the morning, one free interval
	mean $\sum_k p_k L_k$		6.26	6.81	8.86		

$\mathcal{R}_1$, $\mathcal{R}_4$ and $\mathcal{R}_5$ share the on/off pattern but differ in MW; the features (undeliverable energy, thermal use) tell them apart. Benchmark schedule for reference: charge 100/0/200/22/0/0 MW, discharge 0/90/0/0/175/25 MW.



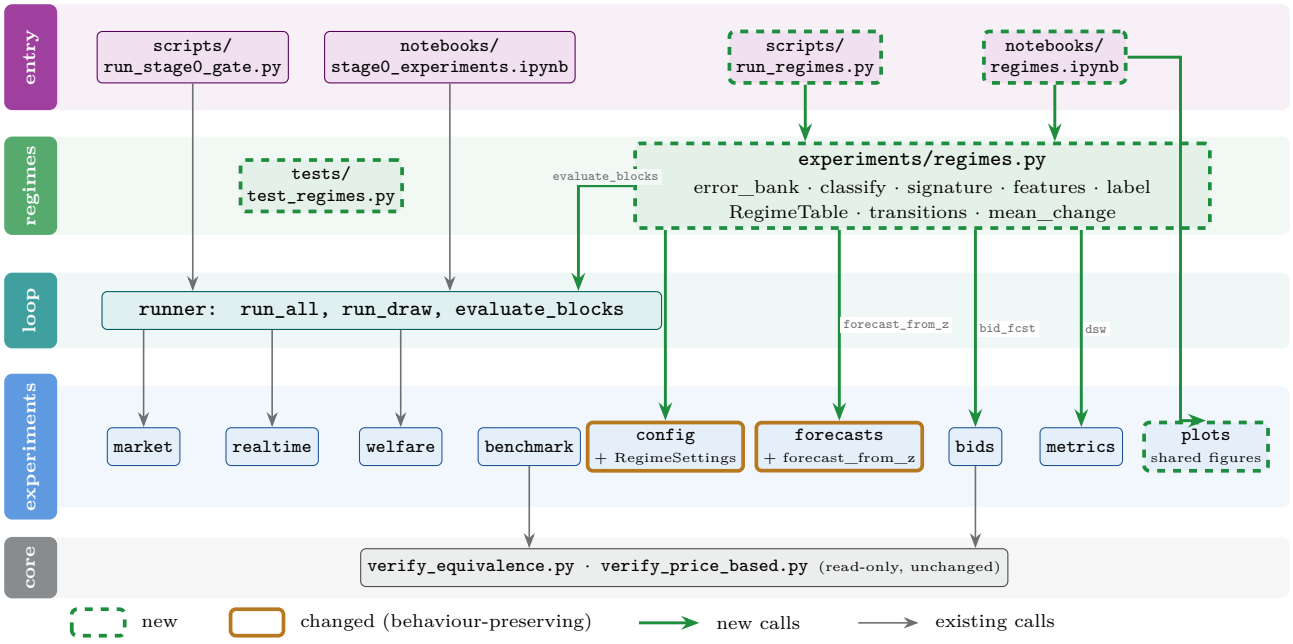
How σ raises the mean loss. Up to $\sigma = 2$ the regime mix is almost unchanged (mean 6.3%). From $\sigma = 5$ on, $\mathcal{R}_1$ (9.3%) loses draws and $\mathcal{R}_5$ (16.5%) gains them: at $\sigma = 20$, $\mathcal{R}_5$ is the most likely single regime. The rise of the mean (6.3% → 8.9%) is mainly this one shift, plus a growing tail of rare regimes ("other").



Where each draw goes (same 400 error shapes at $\sigma = 1$ and $\sigma = 5$). 88% of draws stay in their regime (diagonal) and contribute nothing. The mean rises by +0.558 points of V, and the flows add up to exactly that: $\mathcal{R}_1 \to \mathcal{R}_5$ (21 draws, +7.2 points each) contributes +0.376, two thirds of the change; $\mathcal{R}_4 \to \mathcal{R}_{11}$ (+0.054), $\mathcal{R}_3 \to \mathcal{R}_{12}, \mathcal{R}_{18}$ (+0.054) and $\mathcal{R}_6 \to \mathcal{R}_{15}$ (+0.033) most of the rest. One flow lowers the loss ($\mathcal{R}_8 \to \mathcal{R}_{19}$, −0.010).

5 Code topology: how it plugs into the existing code

The regime analysis is a *reader* of the existing pipeline: it reuses bid construction, clearing, real-time repair and welfare unchanged (`runner.evaluate_blocks`) and adds one module, one script, one notebook and one test file. Two existing modules get small, behaviour-preserving changes.

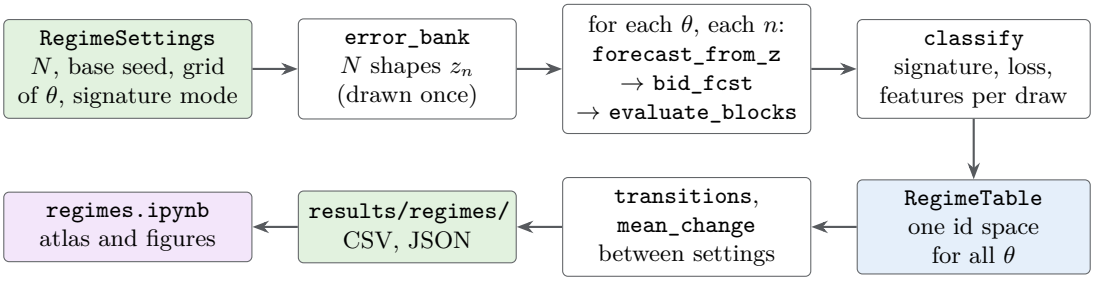


The new **regimes** layer sits between the entry points and the experiment modules. It calls `forecasts` (shapes → forecasts), `bids` (the `B_fcst` bid), `runner.evaluate_blocks` (clearing, real time, welfare: unchanged) and `metrics.dsw`. `plots` collects the figure helpers that today live inside the stage-0 notebook (it is already foreseen in `experiments.md` §3), so both notebooks share them.

5.1 What is new and what changes

File	Role	Main objects
new		
`experiments/regimes.py`	Error-shape bank, classification of draws into regimes, features and labels, regime tables, transitions, decomposition of mean changes.	`error_bank`, `classify`, `signature`, `features`, `label`, `RegimeTable`, `transitions`, `mean_change`
`experiments/plots.py`	Figure helpers shared by the notebooks (energy balance, SOC axes, bid bands, the eight-panel draw, regime charts).	`energy_balance`, `draw_panels`, `regime_bars`, `transition_matrix`
`scripts/run_regimes.py`	Runs a grid of settings on one bank, writes results, prints the regime table.	`main`
`notebooks/regimes.ipynb`	The regime atlas: table, probability chart, transitions, one picture per regime, a regime explorer.	–
`tests/test_regimes.py`	Acceptance tests (§5.4).	–
`results/regimes/`	One CSV row per (setting, draw) with regime id and features; `regimes.json` with the regime dictionary; `meta.json`.	git-ignored
changed, behaviour-preserving		
`experiments/forecasts.py`	Split `forecast_prices` into "draw the shapes" and "apply θ to a shape"; `forecast_prices` calls both and returns identical numbers.	`error_shapes`, `forecast_from_z`
`experiments/config.py`	`RegimeSettings`. (Stable per-draw seeds from (`base_seed`, `draw`) only, i.e. common random numbers and reproducible draws: **done** on 28 Oct 2026, `RunSettings.seed`.)	`RegimeSettings`

5.2 Data flow of one regime run



Cost: one draw takes about 0.03 s, so $N = 500$ draws per setting is 15 s, and the 324-setting stage-1 grid about 80 minutes; the regime layer itself only groups numbers (negligible).

5.3 Interfaces (sketch)

```
# experiments/forecasts.py  (split; forecast_prices returns the same numbers as today)
def error_shapes(T, error, n_ess, rng) -> np.ndarray        # z, shape (n_ess, T)
def forecast_from_z(lam_star, z, error) -> np.ndarray       # max(floor, lam* + bias + sigma*z)

# experiments/regimes.py
def error_bank(n, T, rho, base_seed) -> np.ndarray          # (n, T), shared by every setting
def signature(outcome, mode="day_ahead") -> tuple           # the regime key
def features(case, outcome) -> dict                         # free energy, infeasible, shed, ...
def label(features) -> str                                  # "missed morning sale", ...
def classify(case, bench, bank, error, bid_rule, rt) -> pd.DataFrame   # one row per draw

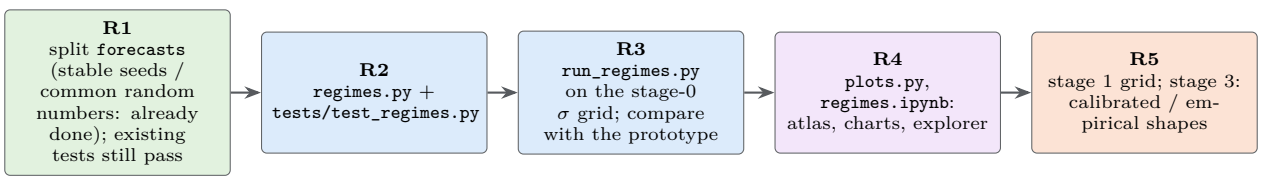
@dataclass
class RegimeTable:                                          # one id space for all settings
    regimes: pd.DataFrame       # id, key, pattern, label, loss, features
    probs: pd.DataFrame         # (setting, id) -> p, se
    def mean_loss(self, setting) -> float                   # sum_k p_k L_k

def transitions(draws_a, draws_b) -> pd.DataFrame           # P_kl on the same bank
def mean_change(draws_a, draws_b, table) -> pd.DataFrame    # flows k -> l and L_l - L_k
```

5.4 Acceptance tests

Test	Passes when
same numbers after the split	`forecast_from_z(error_shapes(...))` equals `forecast_prices` for the same seed, bit for bit
probabilities	$\sum_k \hat{p}_k = 1$ for every setting
exact decomposition	$\sum_k \hat{p}_k L_k$ equals the Monte Carlo mean of the loss to 10^{-9}
regime constant	in `day_case`, the loss spread within every regime is below 10^{-9} (else the coarse-signature mode is required)
zero error	at $\sigma = 0$ there is one regime, with the `B_star_p` loss (6.32% of V)
transitions	row sums of P equal $\hat{p}(\theta)$, column sums equal $\hat{p}(\theta')$; $\sum_{k,l} P_{kl}(L_l - L_k) = \hat{\mu}(\theta') - \hat{\mu}(\theta)$
reproducible	the same `base_seed` gives the same bank and the same table in a new Python session

5.5 Work plan



6 Limitations to state

- **Model probabilities.** Until stage 3, p_k are probabilities under a Gaussian AR(1) error model, not observed frequencies.

- **Test-system specific.** The few, crisp regimes come from the ties of this 6-interval case; a 24-hour system will need the coarse-signature mode and will show fuzzier clusters.
- **Signature choice.** Grouping by the day-ahead battery schedule is exact here (the loss is a regime constant); in larger systems the within-regime spread must be reported.
- **Rare regimes.** With N draws, regimes of probability below about $3/N$ may be missed; the "other" share and $(1-p)^N$ say how much.

Reproducing the prototype. From the `Code/` folder, run `docs/prototypes/regimes_prototype.py` with the `DTU_SYS` Python. It writes the figures `docs/figures/reg_*.pdf` and the numbers `reg_summary.json` (about 4 minutes).